🫀 Synheart — Human Activity Recognition Pipeline

End-to-End: Data → CNN Training → ONNX Export

This notebook walks through the complete HAR pipeline:

Step	What happens
1	Setup — mount Drive, install dependencies
2	Data exploration — UCI-HAR dataset
3	Signal visualisation — raw IMU and gravity separation
4	Model architecture — 1D-CNN design
5	LOSO cross-validation — unbiased accuracy estimate
6	Final model training — with SO(3) orientation augmentation
7	Evaluation — confusion matrix and per-class metrics
8	ONNX export — portable model for iOS/Android/desktop
9	Inference demo — test on held-out windows

Runtime: Set to **T4 GPU** (Runtime → Change runtime type → T4 GPU)

Step 1 — Setup

Mount Google Drive where you uploaded the `activity_state/` folder, then install dependencies.

```
from google.colab import drive
drive.mount('/content/drive')

import os, sys

# ── Change this path to wherever you uploaded activity_state/ in your Drive ──
PROJECT_ROOT = '/content/drive/MyDrive/activity_state'

os.chdir(PROJECT_ROOT)
sys.path.insert(0, PROJECT_ROOT)
print('Working directory:', os.getcwd())
print('Contents:', os.listdir('.'))
```

```
!pip install -q torch torchvision scipy scikit-learn joblib numpy onnx onnxruntime

import torch
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'PyTorch {torch.__version__}  |  Device: {device}')
if device == 'cuda':
    print(f'GPU: {torch.cuda.get_device_name(0)}')
    print(f'VRAM: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB')
```

Step 2 — Data Exploration

UCI-HAR Dataset

30 subjects, phone at waist, **50 Hz** sampling rate.
Each window: **128 samples = 2.56 seconds** (power-of-2 for FFT; captures ≥2 full gait cycles at walking cadence).
50% overlap → new inference every **1.28 seconds**.

The dataset provides pre-separated signals:

- `body_acc_xyz` — gravity removed (Butterworth 0.3 Hz lowpass)
- `body_gyro_xyz` — angular velocity
- `total_acc_xyz` — raw acceleration (used to recover `gravity_mean = total_acc - body_acc`)

Why gravity separation matters: `total_acc` mixes body motion with phone orientation.
`body_acc` is orientation-independent — walking looks the same regardless of which way the phone faces.

```
import numpy as np
from pathlib import Path

DATA_DIR = Path('data/UCI HAR Dataset')
```

```
CLASS_NAMES = ['walking', 'ascending', 'descending', 'sedentary', 'standing', 'lying']
# UCI-HAR original label IDs (1-indexed) → our class names
LABEL_MAP = {1: 'walking', 2: 'ascending', 3: 'descending',
             4: 'sedentary', 5: 'standing', 6: 'lying'}

def load_split(split='train'):
    path = DATA_DIR / split
    signals = path / 'Inertial Signals'

    body_acc_x = np.loadtxt(signals / f'body_acc_x_{split}.txt')
    body_acc_y = np.loadtxt(signals / f'body_acc_y_{split}.txt')
    body_acc_z = np.loadtxt(signals / f'body_acc_z_{split}.txt')
    body_gyr_x = np.loadtxt(signals / f'body_gyro_x_{split}.txt')
    body_gyr_y = np.loadtxt(signals / f'body_gyro_y_{split}.txt')
    body_gyr_z = np.loadtxt(signals / f'body_gyro_z_{split}.txt')
    total_acc_x = np.loadtxt(signals / f'total_acc_x_{split}.txt')
    total_acc_y = np.loadtxt(signals / f'total_acc_y_{split}.txt')
    total_acc_z = np.loadtxt(signals / f'total_acc_z_{split}.txt')

    # Stack signal: (N, 128, 6)
    X_sig = np.stack([
        body_acc_x, body_acc_y, body_acc_z,
        body_gyr_x, body_gyr_y, body_gyr_z
    ], axis=2).astype(np.float32)

    # gravity_mean = total_acc - body_acc, averaged over window → (N, 3)
    gravity_x = (total_acc_x - body_acc_x).mean(axis=1)
    gravity_y = (total_acc_y - body_acc_y).mean(axis=1)
    gravity_z = (total_acc_z - body_acc_z).mean(axis=1)
    X_grav = np.stack([gravity_x, gravity_y, gravity_z], axis=1).astype(np.float32)

    y_raw = np.loadtxt(DATA_DIR / split / f'y_{split}.txt', dtype=int)
    y = np.array([LABEL_MAP[i] for i in y_raw])

    subjects = np.loadtxt(DATA_DIR / split / f'subject_{split}.txt', dtype=int)

    return X_sig, X_grav, y, subjects

X_sig_tr, X_grav_tr, y_tr, subj_tr = load_split('train')
X_sig_te, X_grav_te, y_te, subj_te = load_split('test')

print(f'Train: {X_sig_tr.shape[0]:,} windows   Test: {X_sig_te.shape[0]:,} windows')
print(f'Signal shape per window: {X_sig_tr.shape[1:]}   Gravity: {X_grav_tr.shape[1:]}')
print(f'Subjects — Train: {sorted(np.unique(subj_tr))}   Test: {sorted(np.unique(subj_te))}')
```

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from collections import Counter

COLORS = {'walking': '#4CAF50', 'ascending': '#2196F3', 'descending': '#FF9800',
          'sedentary': '#9C27B0', 'standing': '#F44336', 'lying': '#00BCD4'}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('UCI-HAR Class Distribution', fontsize=14, fontweight='bold')

for ax, (split_y, title) in zip(axes, [(y_tr, 'Train set'), (y_te, 'Test set')]):
    counts = Counter(split_y)
    labels = [c for c in CLASS_NAMES if c in counts]
    values = [counts[c] for c in labels]
    colors = [COLORS[c] for c in labels]
    bars = ax.bar(labels, values, color=colors, edgecolor='white', linewidth=0.8)
    for bar, val in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 15,
                str(val), ha='center', va='bottom', fontsize=10)
    ax.set_title(title, fontsize=12)
    ax.set_ylabel('Windows')
    ax.set_ylim(0, max(values) * 1.15)
    ax.tick_params(axis='x', rotation=20)
    ax.spines[['top','right']].set_visible(False)

plt.tight_layout()
plt.show()
print('\nInsight: Classes are roughly balanced (~800-1400 windows each).')
print('Ascending and descending have fewer windows — stair sequences are shorter in the protocol.')
```

Step 3 — Signal Visualisation

What does body_acc look like for each activity?

The CNN reads **shape** — the temporal pattern of the signal.

- **Walking / ascending / descending** → periodic oscillation at 1–2 Hz
- **Sedentary / standing** → near-flat, near-zero after gravity removal
- **Lying** → near-flat but gravity_mean direction reveals body is horizontal

```
t = np.linspace(0, 2.56, 128)
fig, axes = plt.subplots(3, 2, figsize=(15, 10))
fig.suptitle('body_acc_x  per activity class  (one representative window)', fontsize=14, fontweight='bold')

for ax, cls in zip(axes.flat, CLASS_NAMES):
    idx = np.where(y_tr == cls)[0]
    # pick median-energy window to avoid outliers
    energies = np.sum(X_sig_tr[idx, :, 0]**2, axis=1)
    med_idx  = idx[np.argsort(energies)[len(energies)//2]]

    ax.plot(t, X_sig_tr[med_idx, :, 0], color=COLORS[cls], linewidth=1.5, label='x')
    ax.plot(t, X_sig_tr[med_idx, :, 1], color=COLORS[cls], linewidth=1.0, alpha=0.5, label='y')
    ax.plot(t, X_sig_tr[med_idx, :, 2], color=COLORS[cls], linewidth=1.0, alpha=0.3, label='z')
    ax.set_title(cls.upper(), color=COLORS[cls], fontweight='bold')
    ax.set_xlabel('Time (s)')
    ax.set_ylabel('body_acc (g)')
    ax.axhline(0, color='black', linewidth=0.5, linestyle='--')
    ax.spines[['top','right']].set_visible(False)
    if cls in ('walking', 'ascending', 'descending'):
        ax.legend(fontsize=8)

plt.tight_layout()
plt.show()
print('\nInsight:')
print('  Dynamic classes (walking/ascending/descending) → clear periodic pattern')
print('  Static classes (sedentary/standing/lying)      → near-flat after gravity removal')
print('  The CNN learns these shapes directly — no feature engineering needed.')
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('gravity_mean per activity class  (mean ± std across all windows)', fontsize=13, fontweight='bold')
axis_labels = ['x (forward)', 'y (up)', 'z (right)']

for ax_i, axis_name in enumerate(axis_labels):
    means = [X_grav_tr[y_tr == cls, ax_i].mean() for cls in CLASS_NAMES]
    stds  = [X_grav_tr[y_tr == cls, ax_i].std()  for cls in CLASS_NAMES]
    colors = [COLORS[c] for c in CLASS_NAMES]
    bars = axes[ax_i].bar(CLASS_NAMES, means, yerr=stds, color=colors,
                          capsize=4, edgecolor='white')
    axes[ax_i].set_title(f'gravity_{axis_name}')
    axes[ax_i].set_ylabel('g')
    axes[ax_i].axhline(0, color='black', linewidth=0.5)
    axes[ax_i].tick_params(axis='x', rotation=30)
    axes[ax_i].spines[['top','right']].set_visible(False)

plt.tight_layout()
plt.show()
print('\nInsight:')
print('  gravity_mean tells the CNN where the gravity vector points in phone coordinates.')
print('  Lying has a distinctive gravity pattern (phone horizontal at waist level in UCI-HAR).')
print('  Standing vs sedentary differ subtly — the CNN must rely on body_acc micro-sway.')
```

Step 4 — Model Architecture

Why a 1D-CNN instead of hand-crafted features?

Classical HAR pipelines compress each window into ~50 statistics (mean, std, spectral peak…).
This **loses the time structure** — the CNN cannot tell if the signal is periodic or flat from statistics alone.

A 1D-CNN reads the **raw waveform directly** through 3 convolutional layers,
then global average pooling collapses time → a 128-D feature vector.
Gravity mean is concatenated before the classification head — providing posture
information without coupling to phone orientation.

```
Signal (128×6) ──► Conv1D(6→64, k=5) ──► BN+ReLU ──► Dropout
               ──► Conv1D(64→128, k=3) ──► BN+ReLU ──► Dropout
               ──► Conv1D(128→128,k=3) ──► BN+ReLU
               ──► GlobalAvgPool → (128,)
```

```
                                                ┐
gravity_mean (3,) ──────────────────────────────┤ Concat → (131,)
                                                ┘
                ──► Linear(131→64) ──► BN+ReLU ──► Dropout
                ──► Linear(64→6)   ──► logits
```

~344,000 parameters — small enough to run on a phone CPU in real time.

```
import torch
import torch.nn as nn

class HAR_CNN(nn.Module):
    def __init__(self, n_classes=6):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(6,   64,  kernel_size=5, padding=2, bias=False),
            nn.BatchNorm1d(64), nn.ReLU(), nn.Dropout(0.1),
            nn.Conv1d(64,  128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.1),
            nn.Conv1d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm1d(128), nn.ReLU(),
        )
        self.head = nn.Sequential(
            nn.Linear(128 + 3, 64, bias=False),
            nn.BatchNorm1d(64), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(64, n_classes),
        )

    def forward(self, signal, gravity_mean):
        x = signal.permute(0, 2, 1)   # (B,128,6) → (B,6,128)
        x = self.conv(x)
        x = x.mean(dim=2)             # global avg pool → (B,128)
        x = torch.cat([x, gravity_mean], dim=1)
        return self.head(x)

model = HAR_CNN(n_classes=6)
total_params = sum(p.numel() for p in model.parameters())
trainable    = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Total parameters : {total_params:,}')
print(f'Trainable        : {trainable:,}')

# Quick forward pass check
dummy_sig  = torch.zeros(4, 128, 6)
dummy_grav = torch.zeros(4, 3)
out = model(dummy_sig, dummy_grav)
print(f'Output shape     : {tuple(out.shape)}  (batch=4, classes=6) ✓')
```

Step 5 — LOSO Cross-Validation

Why Leave-One-Subject-Out?

Random train/test splits can leak subject-specific gait patterns into both sets.
LOSO trains on **all subjects except one**, tests on that one — 30 times.
This gives an **unbiased estimate** of how the model generalises to a new person it has never seen.

Important: Early stopping uses 10% of the *training* subjects as a validation set —
never the test subject. Using the test subject for early stopping would be data leakage.

```
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import copy, time

DEVICE       = 'cuda' if torch.cuda.is_available() else 'cpu'
LOSO_EPOCHS  = 35
LOSO_PATIENCE= 7
RANDOM_STATE = 42

# Combine train+test for LOSO
X_sig_all  = np.concatenate([X_sig_tr,  X_sig_te],  axis=0)
X_grav_all = np.concatenate([X_grav_tr, X_grav_te], axis=0)
y_all      = np.concatenate([y_tr,      y_te],      axis=0)
subj_all   = np.concatenate([subj_tr,   subj_te],   axis=0)

le = LabelEncoder()
y_enc = le.fit_transform(y_all)
n_classes = len(le.classes_)
```

```
print('Classes:', list(le.classes_))

def make_loaders(X_sig, X_grav, y_enc, batch=256, shuffle=True):
    ds = torch.utils.data.TensorDataset(
        torch.tensor(X_sig),
        torch.tensor(X_grav),
        torch.tensor(y_enc, dtype=torch.long)
    )
    return torch.utils.data.DataLoader(ds, batch_size=batch, shuffle=shuffle)

def fit(model, X_sig_tr, X_grav_tr, y_tr,
            X_sig_va, X_grav_va, y_va,
            epochs=LOSO_EPOCHS, patience=LOSO_PATIENCE, verbose=False):
    model = model.to(DEVICE)
    opt   = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
    crit  = nn.CrossEntropyLoss()
    tr_loader = make_loaders(X_sig_tr, X_grav_tr, y_tr)
    best_val, best_state, wait = 0.0, None, 0
    history = {'train_loss': [], 'val_acc': []}

    for epoch in range(epochs):
        model.train()
        total_loss = 0
        for sig, grav, labels in tr_loader:
            sig, grav, labels = sig.to(DEVICE), grav.to(DEVICE), labels.to(DEVICE)
            opt.zero_grad()
            loss = crit(model(sig, grav), labels)
            loss.backward()
            opt.step()
            total_loss += loss.item()
        sched.step()

        model.eval()
        with torch.no_grad():
            va_sig  = torch.tensor(X_sig_va).to(DEVICE)
            va_grav = torch.tensor(X_grav_va).to(DEVICE)
            preds   = model(va_sig, va_grav).argmax(1).cpu().numpy()
        val_acc = accuracy_score(y_va, preds)
        history['train_loss'].append(total_loss / len(tr_loader))
        history['val_acc'].append(val_acc)

        if val_acc > best_val:
            best_val   = val_acc
            best_state = copy.deepcopy(model.state_dict())
            wait = 0
        else:
            wait += 1
            if wait >= patience:
                if verbose: print(f'  Early stop at epoch {epoch+1}')
                break

    model.load_state_dict(best_state)
    return model, history

# ── LOSO loop ──────────────────────────────────────────────────────────
subjects   = sorted(np.unique(subj_all))
fold_accs  = []
t0 = time.time()

for i, test_subj in enumerate(subjects):
    te_m = subj_all == test_subj
    tr_m = ~te_m

    tr_idx = np.where(tr_m)[0]
    rng    = np.random.default_rng(RANDOM_STATE + i)
    rng.shuffle(tr_idx)
    n_val       = max(1, len(tr_idx) // 10)
    val_idx     = tr_idx[:n_val]
    pure_tr_idx = tr_idx[n_val:]

    fold_model = HAR_CNN(n_classes).to(DEVICE)
    fold_model, _ = fit(
        fold_model,
        X_sig_all[pure_tr_idx], X_grav_all[pure_tr_idx], y_enc[pure_tr_idx],
        X_sig_all[val_idx],     X_grav_all[val_idx],     y_enc[val_idx],
    )

    fold_model.eval()
    with torch.no_grad():
        te_sig  = torch.tensor(X_sig_all[te_m]).to(DEVICE)
        te_grav = torch.tensor(X_grav_all[te_m]).to(DEVICE)
```

```
        preds   = fold_model(te_sig, te_grav).argmax(1).cpu().numpy()
    acc = accuracy_score(y_enc[te_m], preds)
    fold_accs.append(acc)

    elapsed = time.time() - t0
    eta     = elapsed / (i+1) * (len(subjects) - i - 1)
    print(f'  Fold {i+1:2d}/30  subj={test_subj:2d}  acc={acc:.4f}  '
          f'elapsed={elapsed/60:.1f}m  ETA={eta/60:.1f}m')

loso_acc = np.mean(fold_accs)
print(f'\n✅  LOSO accuracy: {loso_acc:.4f}  ({loso_acc*100:.2f}%)')
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('LOSO Cross-Validation Results', fontsize=13, fontweight='bold')

# Per-fold bar chart
ax = axes[0]
bar_colors = ['#4CAF50' if a >= loso_acc else '#FF9800' for a in fold_accs]
bars = ax.bar(subjects, fold_accs, color=bar_colors, edgecolor='white')
ax.axhline(loso_acc, color='red', linewidth=1.5, linestyle='--', label=f'Mean {loso_acc:.3f}')
ax.set_xlabel('Subject ID')
ax.set_ylabel('Accuracy')
ax.set_title('Per-fold Accuracy')
ax.set_ylim(0.8, 1.01)
ax.legend()
ax.spines[['top','right']].set_visible(False)

# Distribution
ax2 = axes[1]
ax2.hist(fold_accs, bins=10, color='#2196F3', edgecolor='white', alpha=0.8)
ax2.axvline(loso_acc, color='red', linewidth=2, linestyle='--', label=f'Mean {loso_acc:.3f}')
ax2.set_xlabel('Accuracy')
ax2.set_ylabel('Count')
ax2.set_title('Accuracy Distribution Across Folds')
ax2.legend()
ax2.spines[['top','right']].set_visible(False)

plt.tight_layout()
plt.show()
print(f'\nMin fold: {min(fold_accs):.4f}   Max fold: {max(fold_accs):.4f}   Std: {np.std(fold_accs):.4f}')
print('Low-accuracy folds indicate subjects whose gait patterns differ from the training population.')
```

Step 6 — Final Model Training

SO(3) Orientation Augmentation

UCI-HAR used a fixed waist placement. Real-world use is hand-held in any orientation.
To make the CNN robust, we apply **random 3D rotations** to every training window:
7 rotations per sample → **8× dataset size**.

This forces the CNN to learn from body_acc motion patterns (orientation-independent)
rather than gravity_mean direction (orientation-specific to UCI-HAR's waist placement).

```
from scipy.spatial.transform import Rotation

def augment_orientations(X_sig, X_grav, y, n_aug=7):
    N = len(X_sig)
    sig_list  = [X_sig]
    grav_list = [X_grav]
    y_list    = [y]
    rng = np.random.default_rng(RANDOM_STATE)

    for i in range(n_aug):
        R = Rotation.random(N, random_state=rng).as_matrix()   # (N,3,3)
        body_acc_r  = np.einsum('nij,nkj->nki', R, X_sig[:,:,:3])
        body_gyro_r = np.einsum('nij,nkj->nki', R, X_sig[:,:,3:])
        grav_r      = np.einsum('nij,nj->ni',   R, X_grav)
        sig_list.append(
            np.concatenate([body_acc_r, body_gyro_r], axis=2).astype(np.float32))
        grav_list.append(grav_r.astype(np.float32))
        y_list.append(y)

    return (np.vstack(sig_list), np.vstack(grav_list),
            np.concatenate(y_list))

# Split: 70% train, 10% val, 20% test (subject-level)
all_subjs   = sorted(np.unique(subj_all))
```

```
rng         = np.random.default_rng(RANDOM_STATE)
rng.shuffle(all_subjs)
n_test      = max(1, int(len(all_subjs) * 0.20))
n_val       = max(1, int(len(all_subjs) * 0.10))
te_subjs    = all_subjs[:n_test]
va_subjs    = all_subjs[n_test:n_test+n_val]
tr_subjs    = all_subjs[n_test+n_val:]

tr_idx = np.isin(subj_all, tr_subjs)
va_idx = np.isin(subj_all, va_subjs)
te_idx = np.isin(subj_all, te_subjs)

print(f'Train subjects: {len(tr_subjs)} ({tr_idx.sum()} windows)')
print(f'Val   subjects: {len(va_subjs)} ({va_idx.sum()} windows)')
print(f'Test  subjects: {len(te_subjs)} ({te_idx.sum()} windows)')

print('\nApplying SO(3) orientation augmentation (7× rotations)...')
X_sig_aug, X_grav_aug, y_enc_aug = augment_orientations(
    X_sig_all[tr_idx], X_grav_all[tr_idx], y_enc[tr_idx], n_aug=7)
print(f'Augmented training set: {X_sig_aug.shape[0]:,} windows  (was {tr_idx.sum():,})')
```

```
final_model = HAR_CNN(n_classes).to(DEVICE)
final_model, history = fit(
    final_model,
    X_sig_aug,          X_grav_aug,          y_enc_aug,
    X_sig_all[va_idx],  X_grav_all[va_idx],  y_enc[va_idx],
    epochs=60, patience=10, verbose=True
)

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
fig.suptitle('Final Model Training Curves', fontsize=13, fontweight='bold')
axes[0].plot(history['train_loss'], color='#2196F3')
axes[0].set_title('Training Loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Cross-entropy loss')
axes[0].spines[['top','right']].set_visible(False)

axes[1].plot(history['val_acc'], color='#4CAF50')
axes[1].axhline(max(history['val_acc']), color='red', linestyle='--',
                label=f"Best: {max(history['val_acc']):.4f}")
axes[1].set_title('Validation Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].spines[['top','right']].set_visible(False)
plt.tight_layout()
plt.show()
```

Step 7 — Evaluation

Evaluate on the held-out **test subjects** — people the model has never seen during training or early stopping.

```
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.colors as mcolors

final_model.eval()
with torch.no_grad():
    te_sig  = torch.tensor(X_sig_all[te_idx]).to(DEVICE)
    te_grav = torch.tensor(X_grav_all[te_idx]).to(DEVICE)
    te_preds = final_model(te_sig, te_grav).argmax(1).cpu().numpy()

te_true  = y_enc[te_idx]
test_acc = accuracy_score(te_true, te_preds)
print(f'Test accuracy: {test_acc:.4f}  ({test_acc*100:.2f}%)')

cm     = confusion_matrix(te_true, te_preds)
cm_pct = cm.astype(float) / cm.sum(axis=1, keepdims=True) * 100
labels = list(le.classes_)

fig, ax = plt.subplots(figsize=(8, 7))
im = ax.imshow(cm_pct, cmap='Blues', vmin=0, vmax=100)
plt.colorbar(im, ax=ax, label='% of true class')

for i in range(len(labels)):
    for j in range(len(labels)):
        color = 'white' if cm_pct[i,j] > 55 else 'black'
        ax.text(j, i, f'{cm_pct[i,j]:.1f}%', ha='center', va='center',
                color=color, fontsize=10)
```

```
ax.set_xticks(range(len(labels)))
ax.set_yticks(range(len(labels)))
ax.set_xticklabels(labels, rotation=30, ha='right')
ax.set_yticklabels(labels)
ax.set_xlabel('Predicted')
ax.set_ylabel('True')
ax.set_title(f'Confusion Matrix — Test Set  (acc={test_acc:.3f})', fontweight='bold')
plt.tight_layout()
plt.show()

print('\nPer-class report:')
print(classification_report(te_true, te_preds, target_names=labels))
```

```
final_model.eval()
with torch.no_grad():
    logits = final_model(te_sig, te_grav).cpu().numpy()

probs      = np.exp(logits - logits.max(axis=1, keepdims=True))
probs     /= probs.sum(axis=1, keepdims=True)
confidence = probs.max(axis=1)
correct    = (te_preds == te_true)

fig, ax = plt.subplots(figsize=(9, 4))
ax.hist(confidence[correct],  bins=30, alpha=0.7, color='#4CAF50', label='Correct')
ax.hist(confidence[~correct], bins=30, alpha=0.7, color='#F44336', label='Wrong')
ax.set_xlabel('Max softmax probability (confidence)')
ax.set_ylabel('Windows')
ax.set_title('Prediction Confidence — Correct vs Wrong', fontweight='bold')
ax.legend()
ax.spines[['top','right']].set_visible(False)
plt.tight_layout()
plt.show()
print(f'\nMean confidence — Correct: {confidence[correct].mean():.3f}   '
      f'Wrong: {confidence[~correct].mean():.3f}')
print('Insight: Wrong predictions tend to have lower confidence — the model knows when it is uncertain.')
```

Step 8 — ONNX Export

ONNX (Open Neural Network Exchange) is a portable model format that runs on:

- **iOS** — via Core ML conversion
- **Android** — via ONNX Runtime Mobile
- **Desktop** — via ONNX Runtime (Python, C++, C#)

The model must be moved to **CPU** before tracing — ONNX tracing does not support MPS/CUDA device ops.

```
import onnx, onnxruntime as ort
from pathlib import Path

ONNX_PATH = Path('models/cnn_model_aug.onnx')
Path('models').mkdir(exist_ok=True)

export_model = copy.deepcopy(final_model).cpu().eval()
dummy_sig    = torch.zeros(1, 128, 6)
dummy_grav   = torch.zeros(1, 3)

torch.onnx.export(
    export_model,
    (dummy_sig, dummy_grav),
    str(ONNX_PATH),
    input_names   = ['signal', 'gravity_mean'],
    output_names  = ['logits'],
    dynamic_axes  = {'signal': {0:'batch'}, 'gravity_mean': {0:'batch'}, 'logits': {0:'batch'}},
    opset_version = 17,
)

onnx.checker.check_model(str(ONNX_PATH))
size_kb = ONNX_PATH.stat().st_size / 1024
print(f'Saved: {ONNX_PATH}  ({size_kb:.1f} KB)')

# Verify: ONNX output matches PyTorch
rng    = np.random.default_rng(0)
X_chk  = rng.standard_normal((8, 128, 6)).astype(np.float32)
G_chk  = rng.standard_normal((8, 3)).astype(np.float32)

with torch.no_grad():
    pt_out = export_model(torch.from_numpy(X_chk), torch.from_numpy(G_chk)).numpy()
```

```
sess    = ort.InferenceSession(str(ONNX_PATH), providers=['CPUExecutionProvider'])
ort_out = sess.run(None, {'signal': X_chk, 'gravity_mean': G_chk})[0]

max_diff = float(np.abs(pt_out - ort_out).max())
status   = '✅ OK' if max_diff < 1e-4 else '⚠️  WARNING'
print(f'Max logit diff PyTorch vs ONNX: {max_diff:.2e}  [{status}]')
```

```
import joblib

# Save weights and metadata
PT_PATH   = Path('models/cnn_model_aug.pt')
META_PATH = Path('models/cnn_metadata_aug.pkl')

torch.save(final_model.state_dict(), str(PT_PATH))

metadata = {
    'classes':       list(le.classes_),
    'loso_accuracy': float(loso_acc),
    'test_accuracy': float(test_acc),
    'n_classes':     n_classes,
    'augmented':     True,
}
joblib.dump(metadata, str(META_PATH))
print(f'Saved: {PT_PATH}  ({PT_PATH.stat().st_size/1024:.1f} KB)')
print(f'Saved: {META_PATH}')
print(f'\nFinal summary:')
print(f'  LOSO accuracy : {loso_acc:.4f}  ({loso_acc*100:.2f}%)')
print(f'  Test accuracy : {test_acc:.4f}  ({test_acc*100:.2f}%)')
print(f'  Classes       : {list(le.classes_)}')
```

Step 9 — Inference Demo

Simulate what happens at runtime: feed individual windows from the test set through the ONNX model and compare predicted vs true label.

```
sess = ort.InferenceSession(str(ONNX_PATH), providers=['CPUExecutionProvider'])

rng     = np.random.default_rng(99)
indices = rng.choice(np.where(te_idx)[0], size=12, replace=False)

fig, axes = plt.subplots(3, 4, figsize=(16, 9))
fig.suptitle('Inference Demo — 12 random test windows', fontsize=13, fontweight='bold')
t = np.linspace(0, 2.56, 128)

for ax, idx in zip(axes.flat, indices):
    sig  = X_sig_all[idx][np.newaxis].astype(np.float32)
    grav = X_grav_all[idx][np.newaxis].astype(np.float32)
    logits = sess.run(None, {'signal': sig, 'gravity_mean': grav})[0][0]
    probs  = np.exp(logits - logits.max())
    probs /= probs.sum()

    pred_idx  = probs.argmax()
    pred_lbl  = le.classes_[pred_idx]
    true_lbl  = le.inverse_transform([y_enc[idx]])[0]
    correct   = pred_lbl == true_lbl

    ax.plot(t, X_sig_all[idx, :, 0],
            color=COLORS.get(true_lbl, '#888'), linewidth=1.5)
    title_color = '#2E7D32' if correct else '#C62828'
    status_icon = '✓' if correct else '✗'
    ax.set_title(
        f'{status_icon} True: {true_lbl}\nPred: {pred_lbl} ({probs[pred_idx]*100:.0f}%)',
        color=title_color, fontsize=8.5, fontweight='bold'
    )
    ax.set_xlabel('s', fontsize=7)
    ax.tick_params(labelsize=6)
    ax.spines[['top','right']].set_visible(False)

plt.tight_layout()
plt.show()
```

```
# ── Download trained models to your local machine ──────────────────────
# Run this cell to download the files directly, or find them in your Drive folder.

from google.colab import files
files.download('models/cnn_model_aug.pt')
```

```
files.download('models/cnn_model_aug.onnx')
files.download('models/cnn_metadata_aug.pkl')
print('Download triggered — check your browser downloads.')
```

Summary

	Value
Model	1D-CNN, ~344K parameters
Input	128×6 signal + 3 gravity values
Classes	walking, ascending, descending, sedentary, standing, lying
LOSO accuracy	see above
Test accuracy	see above
Output formats	`.pt` (PyTorch), `.onnx` (portable)
Augmentation	SO(3) random rotations × 7 → 8× dataset

Next steps

- **iOS**: convert ONNX → Core ML with `coremltools`
- **Android**: use `cnn_model_aug.onnx` with ONNX Runtime Mobile
- **Real-time inference**: run `src/infer.py --ip <phone-ip> --model cnn --weights aug`
- **Improve sitting/standing**: collect hand-held data with labelled sitting/standing sessions